

Lab 9 – Solutions

November 25, 2024

This lab deals with trigonometric and least square method.

1 Trigonometric interpolation

Consider the case of a periodic function $f(x)$ with period 2π and with $f(0) = f(2\pi)$. The $n + 1$ values at equally spaced nodes

$$x_j = 2\pi j / (n + 1) \quad j = 0, \dots, n$$

are known. The constant step size is

$$h = 2\pi / (n + 1)$$

. The Lagrange interpolator may not approximate well far from the nodes, as it is a polynomial of degree n and generally not periodic.

Goal: Construct a periodic interpolator $\tilde{f}(x)$.

The trigonometric interpolant $\tilde{f}$ is obtained by a linear combination of sines and cosines. Setting $M = \frac{n}{2}$ with n even:

$$\tilde{f}(x) = \frac{a_0}{2} + \sum_{k=1}^M (a_k \cos(kx) + b_k \sin(kx))$$

with complex unknown coefficients a_k for $k = 0, \dots, M$ and b_k for $k = 1, \dots, M$. $\tilde{f}(x)$ is a 2π -periodic function characterized by $M + 1$ frequencies $k = 0, \dots, M$ and coefficients a_k and b_k .

Using the Euler notation, the trigonometric interpolant can be written as:

$$\tilde{f}(x) = \sum_{k=-M}^M c_k e^{ikx}$$

with $c_k = \frac{1}{2}(a_k - ib_k)$ and $c_{-k} = \frac{1}{2}(a_k + ib_k)$ for $k = 0, \dots, M$.

The coefficients c_k are complex and the interpolant is a linear combination of complex exponentials.

The interpolation conditions are:

$$\tilde{f}(x_j) = \sum_{k=-M}^M c_k e^{ikjh} = f(x_j) \quad \text{for } j = 0, \dots, n$$

which gives $n + 1$ equations for the $2M + 1$ unknowns c_k .

With some algebra, the coefficients c_k can be expressed in terms of the $f(x_j)$ (obtaining the **DFT**, Discrete Fourier Transform):

$$c_k = \frac{1}{n+1} \sum_{j=0}^n f(x_j) e^{-ikjh} \quad \text{for } k = -M, \dots, M$$

or in matrix form:

$$\begin{pmatrix} c_{-M} \\ \vdots \\ c_M \end{pmatrix} = \frac{1}{n+1} \begin{pmatrix} e^{iMh0} & \dots & e^{iMhn} \\ \vdots & \ddots & \vdots \\ e^{-iMh0} & \dots & e^{-iMhn} \end{pmatrix} \begin{pmatrix} f(x_0) \\ \vdots \\ f(x_n) \end{pmatrix}$$

$$\mathbf{c} = \mathbb{T}\mathbf{f}$$

where $\mathbf{c} \in \mathcal{R}^{n+1}$, $\mathbf{f} \in \mathcal{R}^{n+1}$ and $\mathbb{T}$ is the Fourier matrix with coefficients:

$$T_{kj} = \frac{1}{n+1} e^{-ikjh} \quad \text{for } k = -M, \dots, M \quad \text{and } j = 0, \dots, n$$

The Inverse Discrete Fourier Transform (IDFT) is:

$$f(x_j) = \sum_{k=-M}^M c_k e^{ikjh} \quad \text{for } j = 0, \dots, n$$

or in matrix form:

$$\mathbf{f} = \mathbb{T}^{-1}\mathbf{c}$$

where $\mathbb{T}^{-1}$ is

$$(T^{-1})_{kj} = e^{ikjh} \quad \text{for } k = -M, \dots, M \quad \text{and } j = 0, \dots, n$$

Theorem 9.1 (Interpolation error). *If f is 2π -periodic and admits continuous and 2π -periodic derivative up to order s included, then the error of the trigonometric interpolant $\tilde{f}$ is bounded by:*

$$\max_{x \in [0, 2\pi]} |f(x) - \tilde{f}(x)| \leq C \frac{1}{n^s}$$

for some constant C . If this holds $\forall s > 0$, then $\tilde{f}$ converges to f exponentially.

Exercise 9.1. a. Implement a function that computes the coefficient of the Fourier trigonometric interpolant.

b. Considering $M = 25$ and evenly distributed nodes in the interval $[0, 2\pi]$, find the trigonometric interpolant of the following functions and study the convergence order of the interpolation motivating the results:

$$f(x) = 1 - \frac{4x^2}{\pi^2} + \frac{4x^3}{\pi^3} - \frac{x^4}{\pi^4}$$

$$f(x) = \text{interp1}([0 \ 2\pi/3 \ 4\pi/3 \ 2\pi], [0 \ 1 \ -1 \ 0], x, 'linear')$$

$$f(x) = 1.0(x > \pi/2 \text{ and } x < 3 * \pi/2)$$

$$f(x) = \sin(x + \cos(5x))$$

Solution Exercise 9.1.

discrete_fourier_transform.m

```
function c=discrete_fourier_transform(xj,fxj,k)
    n=numel(k)-1;
    T = (1/(n+1)) * exp (-i .* (k * xj) );
    c = T * fxj;
```

end

ex_9_1.m

```
%% Exercise 9.1
clear all; close all; clc;

M = 25; k = [-M:M]';
n = 2*M;
jj = [0:n];
h = 2*pi/(n+1);
xj = jj*h;
```

```

%% First Function f1
f1 = @(x) 1-(4*x.^2)/pi^2 + (4*x.^3)/pi^3 - x.^4/pi^4;
% Compute the coefficients for the trigonometric interpolation

c=discrete_fourier_transform(xj, f1(xj)',k);

a = c(M+2:end) + c(M:-1:1);
b = i * (c(M+2:end) - c(M:-1:1));
a0= c(M+1) + c(M+1);
% Compute the interpolant
x = linspace (0, 2*pi, 1000).';

F = a0/2;
for ii = 1 : M
    Fkk= a(ii) * cos (ii * x) + b(ii) * sin (ii * x);
    F=F+Fkk;
end
figure
plot (x, F,'LineWidth',2)
hold on
plot(x, f1(x), xj, f1(xj), 'x')
title("f=1- $\frac{4x^2}{\pi^2} + \frac{4x^3}{\pi^3} - \frac{x^4}{\pi^4}$ ", 'Interpreter','latex','FontSize',18)
legend('trigonometric interpolant', 'true function','nodes values','Location','north')

%Convergence study
err=[];

for iM=1:6
    M=2.^iM;
    n=2*M;
    k = [-M:M]';
    jj = [0:n];
    h = 2*pi/(n+1);
    xj = jj*h;
    c=discrete_fourier_transform(xj, f1(xj)',k);

    a = c(M+2:end) + c(M:-1:1);
    b = i * (c(M+2:end) - c(M:-1:1));
    a0= c(M+1) + c(M+1);
    % Compute the interpolant
    x = linspace (0, 2*pi, 1000).';

    F = a0/2;
    for ii = 1 : M
        Fkk= a(ii) * cos (ii * x) + b(ii) * sin (ii * x);
        F=F+Fkk;
    end
    err=[err norm(f1(x)-F, inf)];
end

figure
nvec=2*2.^[1:6];
loglog(nvec, err)
hold on
loglog(nvec, 1./nvec.^2)
legend("err", 'O(n^-2)')
%% continuous function
f2 = @(x) interp1 ([0 2*pi/3 4*pi/3 2*pi], [0 1 -1 0], x, 'linear');
% Compute the coefficients for the trigonometric interpolation

c=discrete_fourier_transform(xj, f2(xj)',k);

a = c(M+2:end) + c(M:-1:1);
b = i * (c(M+2:end) - c(M:-1:1));
a0= c(M+1) + c(M+1);
% Compute the interpolant
x = linspace (0, 2*pi, 1000).';

F = a0/2;

```

```

for ii = 1 : M
    Fkk= a(ii) * cos (ii * x) + b(ii) * sin (ii * x);
    F=F+Fkk;
end
figure
plot (x, F,'LineWidth',2)
hold on
plot(x, f2(x), xj, f2(xj), 'x')
title("non derivable function", 'Interpreter','latex','FontSize',18)
legend('trigonometric interpolant', 'true function','nodes values','Location','southwest')

%Convergence study
err=[];

for iM=1:6
    M=2.^iM;
    n=2*M;
    k = [-M:M]';
    jj = [0:n];
    h = 2*pi/(n+1);
    xj = jj*h;
    c=discrete_fourier_transform(xj, f2(xj)',k);

    a = c(M+2:end) + c(M:-1:1);
    b = i * (c(M+2:end) - c(M:-1:1));
    a0= c(M+1) + c(M+1);
    % Compute the interpolant
    x = linspace (0, 2*pi, 1000).';

    F = a0/2;
    for ii = 1 : M
        Fkk= a(ii) * cos (ii * x) + b(ii) * sin (ii * x);
        F=F+Fkk;
    end
    err=[err max(abs(f2(x)-F))];

end

figure
nvec=2*2.^[1:6];
loglog(nvec, err)
hold on
loglog(nvec, 1./nvec)
legend("err", 'O(n)')
%% step function
f3 = @(x) 1.0 .* (x > pi/2 & x < 3*pi/2);
% Compute the coefficients for the trigonometric itnerpolation

c=discrete_fourier_transform(xj, f3(xj)',k);

a = c(M+2:end) + c(M:-1:1);
b = i * (c(M+2:end) - c(M:-1:1));
a0= c(M+1) + c(M+1);
% Compute the interpolant
x = linspace (0, 2*pi, 1000).';

F = a0/2;
for ii = 1 : M
    Fkk= a(ii) * cos (ii * x) + b(ii) * sin (ii * x);
    F=F+Fkk;
end
figure
plot (x, F,'LineWidth',2)
hold on
plot(x, f3(x), xj, f3(xj), 'x')
title("f=1-\frac{4x^2}{\pi^2}+\frac{4x^3}{\pi^3}-\frac{x^4}{\pi^4}", 'Interpreter','latex','FontSize',18)
legend('trigonometric interpolant', 'true function','nodes values','Location','south')
legend("Position", [0.33388,0.14291,0.28939,0.095918])

% Property editing

```

```

positionPropObjs = findobj(gcf, "-property", "Position");
positionPropObjs(2).Position = [0.3339 0.1429 0.2894, 0.0959];
%Convergence study
err=[];

for iM=1:6
    M=2.^iM;
    n=2*M;
    k = [-M:M]';
    jj = [0:n];
    h = 2*pi/(n+1);
    xj = jj*h;
    c=discrete_fourier_transform(xj, f3(xj)',k);

    a = c(M+2:end) + c(M:-1:1);
    b = i * (c(M+2:end) - c(M:-1:1));
    a0= c(M+1) + c(M+1);
    % Compute the interpolant
    x = linspace (0, 2*pi, 1000).';

    F = a0/2;
    for ii = 1 : M
        Fkk= a(ii) * cos (ii * x) + b(ii) * sin (ii * x);
        F=F+Fkk;
    end
    err=[err max(abs(f3(x)-F))];

end

figure
nvec=2*2.^[1:6];
loglog(nvec, err)
hold on
loglog(nvec, 1./nvec)
legend("err", 'O(n)', 'Location', 'southwest')

%% C^\infty function
f4 = @(x) sin (x + cos (5*x));
% Compute the coefficients for the trigonometric itnerpolation

c=discrete_fourier_transform(xj, f4(xj)',k);

a = c(M+2:end) + c(M:-1:1);
b = i * (c(M+2:end) - c(M:-1:1));
a0= c(M+1) + c(M+1);
% Compute the interpolant
x = linspace (0, 2*pi, 1000).';

F = a0/2;
for ii = 1 : M
    Fkk= a(ii) * cos (ii * x) + b(ii) * sin (ii * x);
    F=F+Fkk;
end
figure
plot (x, F, 'LineWidth',2)
hold on
plot(x, f4(x), xj, f4(xj), 'x')
title("f=sin(x + cos(5x))", 'Interpreter','latex','FontSize',18)
legend('trigonometric interpolant', 'true function','nodes values','Location','southwest')

%Convergence study
err=[];
for iM=1:6
    M=2.^iM;
    n=2*M;
    k = [-M:M]';
    jj = [0:n];
    h = 2*pi/(n+1);
    xj = jj*h;
    c=discrete_fourier_transform(xj, f4(xj)',k);

```

```

a = c(M+2:end) + c(M:-1:1);
b = i * (c(M+2:end) - c(M:-1:1));
a0= c(M+1) + c(M+1);
% Compute the interpolant
x = linspace (0, 2*pi, 1000).';

F = a0/2;
for ii = 1 : M
    Fkk= a(ii) * cos (ii * x) + b(ii) * sin (ii * x);
    F=F+Fkk;
end
err=[err norm((f4(x)-F),inf)];

end

figure
nvec=2*2.^[1:6];
loglog(nvec, err)
hold on
loglog(nvec, exp(-0.2*nvec))
legend("err", 'O(exp(-n))')

```

Exercise 9.2. Repeat point b of the previous exercise using MATLAB command `interpft`.

Solution Exercise 9.2.

ex_9_2.m

```

%% Exercise 9.2
clear all; close all; clc;

%% First Function f1
f1 = @(x) 1-(4*x.^2)/pi^2 + (4*x.^3)/pi^3 - x.^4/pi^4;
n = 50;
jj = [0:n];
h = 2*pi/(n+1);
xj = jj*h;

F=interpft(f1(xj),1000);
x = linspace (0, 2*pi, 1000).';
figure
plot (x, F,'LineWidth',2)
hold on
plot(x, f1(x), xj, f1(xj), 'x')
title("f=1 - \frac{4x^2}{\pi^2} + \frac{4x^3}{\pi^3} - \frac{x^4}{\pi^4}", 'Interpreter','latex','FontSize',18)
legend('trigonometric interpolant', 'true function','nodes values','Location','north')

%Convergence study
err=[];
for iM=1:6
    M=2.^iM;
    n=2*M;
    jj = [0:n];
    h = 2*pi/(n+1);
    xj = jj*h;
    F=interpft(f1(xj),1000);
    x = linspace (0, 2*pi, 1000);
    err=[err norm(f1(x)-F,inf)];
end

figure
nvec=2*2.^[1:6];
loglog(nvec, err)
hold on
loglog(nvec, 1./(nvec.^2))

```

```

legend("err", 'O(n^-2)')
%% continuous function
f2 = @(x) interp1 ([0 2*pi/3 4*pi/3 2*pi], [0 1 -1 0], x, 'linear');
% Compute the coefficients for the trigonometric interpolation
n = 50;
jj = [0:n];
h = 2*pi/(n+1);
xj = jj*h;

F=interpft(f2(xj),1000);
x = linspace (0, 2*pi, 1000).';
figure
plot (x, F,'LineWidth',2)
hold on
plot(x, f2(x), xj, f2(xj), 'x')
title("non derivable function", 'Interpreter','latex','FontSize',18)
legend('trigonometric interpolant', 'true function','nodes values','Location','southeast')

%Convergence study
err=[];
for iM=1:6
    M=2.^iM;
    n=2*M;
    jj = [0:n];
    h = 2*pi/(n+1);
    xj = jj*h;
    F=interpft(f2(xj),1000);
    x = linspace (0, 2*pi, 1000);
    err=[err norm(f2(x)-F, inf)];
end

figure
nvec=2*2.^[1:6];
loglog(nvec, err)
hold on
loglog(nvec, 1./(nvec))
legend("err", 'O(n)')
%% step function
f3 = @(x) 1.0 .* (x > pi/2 & x < 3*pi/2);
n = 50;
jj = [0:n];
h = 2*pi/(n+1);
xj = jj*h;

F=interpft(f3(xj),1000);
x = linspace (0, 2*pi, 1000).';
figure
plot (x, F,'LineWidth',2)
hold on
plot(x, f3(x), xj, f3(xj), 'x')
title("f = 1.0(x > pi/2 and x < 3pi/2)", 'Interpreter','latex','FontSize',18)
legend('trigonometric interpolant', 'true function','nodes values','Location','south')
% Property editing
positionPropObjs = findobj(gcf, "-property", "Position");
positionPropObjs(2).Position = [0.3339 0.1429 0.2894, 0.0959];
%Convergence study
err=[];
for iM=1:6
    M=2.^iM;
    n=2*M;
    jj = [0:n];
    h = 2*pi/(n+1);
    xj = jj*h;
    F=interpft(f3(xj),1000);
    x = linspace (0, 2*pi, 1000).';
    err=[err max(abs(f3(x)-F))];
end

```

```

end

figure
nvec=2*2.^[1:6];
loglog(nvec, err)
hold on
loglog(nvec, 1./(nvec))
legend("err", 'O(n)')
%% step function
f4 = @(x) sin (x + cos (5*x));
n = 50;
jj = [0:n];
h = 2*pi/(n+1);
xj = jj*h;

F=interpft(f4(xj),1000);
x = linspace (0, 2*pi, 1000).';
figure
plot (x, F,'LineWidth',2)
hold on
plot(x, f4(x), xj, f4(xj), 'x')
title("f=sin(x + cos(5x))", 'Interpreter','latex','FontSize',18)
legend('trigonometric interpolant', 'true function','nodes values','Location','southwest')

%Convergence study
err=[];
for iM=1:6
    M=2.^iM;
    n=2*M;
    jj = [0:n];
    h = 2*pi/(n+1);
    xj = jj*h;
    F=interpft(f4(xj),1000);
    x = linspace (0, 2*pi, 1000);
    err=[err norm(f4(x)-F,inf)];
end

figure
nvec=2*2.^[1:6];
loglog(nvec, err)
hold on
loglog(nvec, exp(-0.2*nvec))
legend("err", 'O(exp(-n))')

```

Exercise 9.3 (Exam exercise). Consider the function

$$f(x) = \cos(5x + \sin(5x))$$

- Use the provided function `int_trigonometrica.m` to compute the Fourier coefficients of the trigonometric interpolant.
- Study the convergence of the trigonometric interpolant to the function $f(x)$. Motivate the results.
- Compare the trigonometric interpolant with the Lagrange interpolant of degree 10 and 15. Motivate the results.

Solution Exercise 9.3.

ex_9_3.m

```

%% Exercise 9.3
clear all; close all; clc;
f = @(x) cos (5*x + sin(5*x));

```



```

n = 50;
I=[0,2*pi];
[~, a,b]=int_trigonometrico(I,n,f);
x = linspace (0, 2*pi, 1000).';
F = a(1)/2;
for ii = 1 : n/2
    Fkk= a(ii+1) * cos (ii * x) + b(ii) * sin (ii * x);
    F=F+Fkk;
end

figure
plot (x, F,'LineWidth',2)
hold on
plot(x, f(x), 'x-')
title("f=cos(5x+sin(5x))", 'Interpreter','latex','FontSize',18)
legend('trigonometric interpolant', 'true function','Location','southwest')

%% Convergence study
err=[];
for iM=1:6
    M=2.^iM;
    n=2*M;
    [~, a,b]=int_trigonometrico(I,n,f);
    x = linspace (0, 2*pi, 1000).';
    F = a(1)/2;
    for ii = 1 : n/2
        Fkk= a(ii+1) * cos (ii * x) + b(ii) * sin (ii * x);
        F=F+Fkk;
    end
    err=[err max(abs(f(x)-F))];
end

figure
nvec=2*2.^[1:6];
loglog(nvec, err)
hold on
loglog(nvec, exp(-0.2*nvec))
legend("err", 'O(exp(-n))')
%% point C
degree=15;
xnodes=linspace(0,2*pi,degree+1);
fnodes=f(xnodes);
pp=polyfit(xnodes,fnodes, degree);
y=polyval(pp, x);
figure
plot (x, F,'LineWidth',2)
hold on
plot(x, y,'LineWidth',2)
plot(x, f(x))
title("f=cos(5x+sin(5x))", 'Interpreter','latex','FontSize',18)
legend('trigonometric interpolant', 'Lagrange Polynomial','true function','Location','southwest')

errLagrange=max(abs(y-f(x)));
errFourier=max(abs(F-f(x)));
[errLagrange errFourier]

```

In the Lagrangian polynomial the interpolant was expressed as a basis function times coefficients that were the function values at the nodes: We can express the trigonometric interpolant in a similar way:

$$\tilde{f}(x) = \sum_{j=0}^n f(x_j) \varphi_j^T(x)$$

with the Lagrangian basis functions:

$$\varphi_j^T(x) = \frac{1}{n+1} \sum_{k=-M}^M e^{ik(x-jh)} \quad \text{for } j = 0, \dots, n$$

Exercise 9.4. Given $n = 4$ and $h = 2\pi/(n+1)$ find the trigonometric Lagrangian basis functions $\varphi_j^T(x)$ for $j = 0, \dots, 4$ and compare it with the Lagrangian basis functions seen in the previous laboratory.

Solution Exercise 9.4.

ex_9_4.m

```
%% Exercise 9.4
clear all; close all

M = 2;
k = -M:M;
n = 2*M;
N = [0:n];
h = 2*pi/(n+1);

xj = N*h;

x = linspace (xj(1), xj(end), 1000).';
figure (1)
phiT=[];
for jj = N
    phit = 1/(n+1)*sum (exp (1i * k' * (x' - h*jj)));
    plot (x, phit)
    phiT=[phiT phit'];
    hold all
end
title ("trigonometric lagrangian basis")

%%
figure (2)
hold on, box on

phi=zeros(numel(xj), numel(xj));
for j = 1:numel(xj)

    nodes = setdiff(xj, xj(j));

    phi(:, j) = poly(nodes) / polyval(poly(nodes), xj(j));
    plot(x, polyval(phi(:, j), x), 'LineWidth', 2)

end

title ("polynomial lagrangian basis")
```

2 Least Square Method

The least square method is used to approximate a function $f(x)$ by a polynomial of degree $m \ll n$, with n the number of nodes. The polynomial is obtained by minimizing the error function in the least square sense:

$$\min_{p \in \mathcal{P}_m} \sum_{i=0}^n (y_j - p(x_j))^2$$

where $\mathcal{P}_m$ is the space of polynomials of degree m . The general form of the polynomial is:

$$p(x) = \sum_{k=0}^m a_k x^k$$

If $m = 1$, the polynomial is called regression line and the error is minimized by the line that passes through the points $(x_j, f(x_j))$. We need to find the coefficients a_0 and a_1 .

The problem can be stated as follows:

$$\Phi(b_0, b_1) = \sum_{j=0}^n (y_j - (b_0 + b_1 x_j))^2$$

Find (a_0, a_1) such that

$$\Phi(a_0, a_1) = \min_{b_0, b_1} \Phi(b_0, b_1)$$

The minimum is found by setting the derivatives of Φ with respect to b_0 and b_1 to zero:

$$\frac{\partial \Phi}{\partial b_0} = 0 \quad \text{and} \quad \frac{\partial \Phi}{\partial b_1} = 0$$

which given the solution:

$$a_0 = \frac{\sum_{j=0}^n y_j \sum_{j=0}^n x_j^2 - \sum_{j=0}^n x_j \sum_{j=0}^n y_j x_j}{D}$$

$$a_1 = \frac{(n+1) \sum_{j=0}^n x_j y_j - \sum_{j=0}^n x_j \sum_{j=0}^n y_j}{D}$$

with $D = (n+1) \sum_{j=0}^n x_j^2 - (\sum_{j=0}^n x_j)^2$.

Exercise 9.5. Generate a random sample of $n+1$ points (x_j, y_j) .

- Compute the coefficients of the regression line. Plot the points and the regression line.
- Repeat the exercise using the MATLAB command `polyfit`.
- Compute the mean square error of the regression line. Verify that the error is minimized.

Solution Exercise 9.5.

ex_9_5.m

```
%% Exercise 9.5
clear all; close all

n = 15;
x = sort (rand (n, 1));
y = 2 + 3*x + randn(n, 1)/10;

figure (1)
plot (x, y, 'x')
hold all

p = polyfit (x, y, 1);

D = (n) * sum (x.^2) - sum (x) ^2;

% a is the same as p
a(1) = 1/D * ((n) * sum (x .* y) - sum (x) * sum(y));
a(2) = 1/D * (sum (x.^2) * sum (y) - sum (x) * sum (x .* y));

plot (x, a(1)*x + a(2))
plot(x, polyval(p, x))
legend ("Data", "Self-Computation", "with polyfit", 'Location', 'southeast')
%% compute mean square error
eta = sum ((a(1)*x + a(2) - y).^2)/n

figure (2)
da = (- a(1)/2) : a(1)/10 : (+ a(1)/2);
aa = a(1) + da;
ii = 1;
for al_a = aa
    eta_a(ii) = sum ((al_a*x + a(2) - y).^2)/n;
    ii = ii+1;
end

plot (aa, eta_a, a(1), eta, 'x')

figure (3)
```

```

da = (- a(2)/2) : a(2)/10 : (+ a(2)/2);
aa = a(2) + da;
ii = 1;
for a2_a = aa
    eta_a(ii) = sum ((a(1)*x + a2_a - y).^2)/n;
    ii = ii+1;
end

plot (aa, eta_a, a(2), eta, 'x', 'MarkerSize',15, 'LineWidth',3)

```